
Big Papers Require Big Humility

Document Number: P4061R0
Date: 2026-03-14
Audience: LEWG, SG1, Direction Group
Reply-to: Vinnie Falco vinnie.falco@gmail.com

Table of Contents

Abstract

Revision History

R0: March 2026 (post-Croydon mailing)

The Responsibility

Your Strongest Domain Is Not Your Only Domain

When a Domain Expert Reports a Problem, That Is a Gift

Build the Hardest Example, Not the Easiest

Expressibility Is Not Fitness

Independent Adoption Is the Only Evidence That Scales

Humility Compounds

Corporate Backing Is a Resource, Not a Vote

The Empty Seat Is Your Responsibility

Ship What You Can Demonstrate

Know Yourself

Acknowledgments

Abstract

Writing a large proposal is an act of ambition and generosity. The author gives years of their life to a design they believe will serve millions of programmers. That ambition deserves respect. This paper is written in that spirit - not to discourage ambition, but to ask that it be accompanied by the self-awareness that makes ambition

trustworthy.

Large proposals shape the standard for decades. The larger the proposal, the greater the author's responsibility - not just to get the engineering right, but to seek disconfirming evidence, engage domain experts whose work the proposal affects, and scope claims to demonstrated capability. This paper offers guidance for authors of large-scope proposals. Every section describes a pattern the author may recognize. The calibration questions are offered as a mirror, not a verdict.

This is the third and final paper in a trilogy. "Big Claims Require Big Evidence" (P4059R0) addresses the committee's process. "Big Reputation Requires Big Responsibility" (P4060R0) addresses the weight of individual standing. This paper addresses the person behind the proposal - the human being whose ambition, conviction, and emotional investment shape the standard for better or worse.

Revision History

R0: March 2026 (post-Croydon mailing)

- Initial version.

The Responsibility

Have I compared my framework to a foundational contribution - the STL, the iterator model, structured programming itself? Do I believe my decomposition is as universal as the one I am comparing it to?

If my framework claims multiple domains, have I demonstrated it in each one - or only in the one I know best?

Large proposals consume years of committee time. Dozens of engineers review the design, debate the wording, implement prototypes, and write national body comments. The committee's investment is proportional to the proposal's scope. The author's responsibility must be proportional too.

That responsibility is not only technical. It is cultural. How the author responds to criticism, engages domain experts, and scopes claims determines whether the proposal serves users or serves the proposal. A framework that claims universality and delivers it is a gift to the language. A framework that claims universality and does not is a cost the committee and its users pay for years.

The guidance in this paper is not hypothetical. Every section describes a pattern that has occurred in the committee's recent history. The patterns are described without names because the lessons are universal. Any author of a large proposal - past, present, or future - may recognize themselves.

Your Strongest Domain Is Not Your Only Domain

Do my examples cover the hardest problems in every domain I claim, or only the easiest problems in my strongest domain?

If I published examples of my framework in use, how many are in the domain I claim but did not build for?

A framework that excels at compile-time work graphs may also claim to handle networking. A framework that excels at GPU dispatch may also claim to handle sequential I/O. The claim is plausible. The framework's abstractions are general. A small example demonstrates that the adjacent domain's operations can be expressed in the framework's vocabulary.

But expressibility is not fitness. A framework fits a domain when it handles that domain's hardest problems without forcing the domain's practitioners to sacrifice properties they depend on. If the domain's practitioners must lose data, bypass the framework's composition model, or convert routine outcomes into exceptions to use the framework, the framework does not fit the domain. It merely expresses it.

The architect who recognizes the boundary between the strong domain and the claimed domain - and names it publicly - builds trust. The architect who publishes seven examples in the strong domain and zero in the claimed domain has named the boundary implicitly. The committee will eventually notice.

When a Domain Expert Reports a Problem, That Is a Gift

When a domain expert published a paper identifying a structural difficulty in my framework, did I investigate or defend?

Did I add a section to my revision history acknowledging the problem - and then move on without resolving it?

How many revision cycles has that acknowledgment survived without resolution?

A domain expert who identifies a structural difficulty in a large proposal is providing the most valuable feedback the proposal can receive. The expert has domain knowledge the author may lack. The expert has seen the problem in production. The expert is telling the author where the boundary is.

The correct response is investigation. Build the expert's example. Reproduce the difficulty. Determine whether it is fundamental or incidental. If it is fundamental, scope the claim. If it is incidental, fix it and show the fix.

The incorrect response is acknowledgment without resolution. A revision history that says "we added a section on X" when X remains unsolved is a promise the committee will treat as fulfilled. The section exists. The problem does not go away. Five revisions later, the section is still there, the problem is still open, and the committee has advanced the proposal on the strength of an acknowledgment that was never a resolution.

Acknowledgment is not resolution. If a structural difficulty survives three revision cycles, it is not a bug awaiting a fix. It is a boundary awaiting a name.

Build the Hardest Example, Not the Easiest

Have I demanded code from my critics while providing no code in their domain?

Is my evidence for a domain claim a toy library that compiles, or a production-scale constructed comparison?

When a critic raises a concern, demanding code is fair. "Show me the problem" is a reasonable request. But the demand is symmetric. If the author asks the critic to demonstrate the problem with code, the author must demonstrate the solution with code in the critic's domain.

A toy library that compiles is not evidence that a framework works at production scale. A small third-party project that expresses a domain's operations in the framework's vocabulary demonstrates expressibility, not fitness. The committee should require more.

A constructed comparison - the best possible implementation in both the framework and the alternative, evaluated on the hardest problems in the claimed domain - is evidence. The comparison table speaks for itself. If the framework handles the hard case cleanly, the claim is substantiated. If every approach in the framework pays a cost the alternative does not, the claim is not.

Build the hardest example. If the hardest example works, the easy ones follow. If the hardest example does not work, the easy ones are misleading.

Expressibility Is Not Fitness

Can I demonstrate my framework in the claimed domain only by writing code that no typical user would write?

If my proof of concept requires expertise that only the framework's authors possess, is it evidence for the framework or evidence for the authors?

Does the user experience of my framework in the claimed domain match the user experience in my strong domain - or does it require a different level of expertise entirely?

A brilliant engineer can bridge any two systems. Given enough expertise, any domain's operations can be expressed in any sufficiently general framework. The bridge compiles. The proof is valid. The conclusion - that the framework fits the domain - does not follow.

Expressibility asks: can the domain's operations be represented in the framework's vocabulary? Fitness asks: can a typical practitioner in that domain use the framework without sacrificing the properties they depend on, at a complexity level comparable to the framework's strong domain? These are different questions. A four-hundred-line adapter that requires intimate knowledge of the framework's operation state model to bridge two async paradigms is an existence proof, not a usability proof.

The danger is that the framework's strongest advocates are often its most capable engineers. They can make anything work. They write the bridge in an afternoon and present it as evidence. The audience sees the bridge and concludes the framework fits. What the audience does not see is that the bridge required skills the audience does not have, and that the domain's practitioners - the people who will actually use the framework in production - would not write that bridge, would not maintain that bridge, and would not trust that bridge.

A proof is not a user. If the framework requires its architects to demonstrate the claimed domain because no one else can, the framework does not fit the domain. It fits the architects.

Independent Adoption Is the Only Evidence That Scales

Is every user of my framework an employee of my employer or a co-author of my paper?

Has anyone outside my organization built a library on my framework? Has anyone built an application on that library?

If I believe most users will use a different interface and only a minority will use my framework's native sub-language, is the minority's experience sufficient evidence for universality?

The author's team built the framework. The author's employer uses it in production. These are necessary but not sufficient. The author's team understands the framework's idioms. The author's employer has access to the author for questions. Neither condition holds for the broader ecosystem.

Independent adoption is the evidence that scales. The adoption ladder has five rungs: the authors built it, independent developers used it, independent libraries were built on top of it, independent applications were built on those libraries, and real users - not employees of the proposing organization - shipped production code. Each rung is evidence that the previous one was not enough.

If the author believes that most users will interact with the framework through a higher-level interface - coroutines, for example - then most users are not exercising the framework's native claims. The evidence base is the minority who use the framework directly. The author should ask whether the minority's experience is sufficient to substantiate a universality claim that affects the majority.

Humility Compounds

When multiple independent papers identify the same structural difficulty in my framework, do I investigate or defend?

Do I engage with the criticism on the reflector, or do I let the revision history carry the acknowledgment while I work on other things?

When one paper identifies a difficulty, it may be a misunderstanding. When two papers identify the same difficulty, it is a signal. When three papers from three independent authors identify the same structural difficulty across three years, it is a finding. The correct response is not defense. It is investigation.

Reputation in the committee is built over decades. It is built by engaging honestly with criticism, by changing course when the evidence warrants it, and by crediting the people who identified the problems. The committee remembers who engaged and who defended. That memory persists across standard cycles. The author who investigates a reported difficulty and says "you are right, this is a boundary" earns more credibility than the author who carries the acknowledgment for five revisions without resolution.

Humility compounds. So does defensiveness. The difference is visible over time.

Corporate Backing Is a Resource, Not a Vote

Does my employer send more engineers to the relevant study groups than any other organization?

Have I sought domain-expert review from practitioners outside my employer?

An employer that funds ten engineers to attend every telecon and every study group meeting is providing an extraordinary resource: implementation capacity, review bandwidth, and institutional continuity. That resource is valuable. It is not a mandate for design direction.

When the room composition reflects one employer's priorities, the committee's decisions reflect those priorities. This is not malicious. Engineers naturally advocate for the designs their employer uses and funds. But the effect is that domain expertise from smaller organizations, independent practitioners, and the users who will actually live with the standard is structurally underrepresented.

The architect who recognizes this dynamic and actively seeks review from outside their employer builds a stronger proposal. The architect who relies on their employer's room presence to advance the proposal builds a fragile one. The standard is a commons. Every employer depends on it. The employer that captures a design direction has won a battle and damaged the field on which all future battles are fought.

The Empty Seat Is Your Responsibility

Has a domain expert stopped attending since the committee chose my framework's direction?

Did I reach out to understand why? Did I ask what would bring them back?

When the most experienced practitioner in a domain stops attending committee meetings, the committee has lost irreplaceable expertise. The practitioner's absence is not a personal failing. It is a signal that the committee's direction has diverged from the domain's needs far enough that continued participation feels futile.

The architect whose framework claimed that domain bears a proportional responsibility. Not sole responsibility - the committee's process, the polls, the room dynamics all contributed. But the architect's scope claim is what brought the framework into the domain. If the domain's foremost expert concluded that the framework cannot serve the domain and stopped participating, the architect should hear that as the strongest possible feedback.

The empty seat is evidence. It is the evidence that no paper can provide and no poll can capture. It is a practitioner's judgment, expressed not in words but in absence, that the committee's direction is not worth their time. That judgment deserves investigation, not dismissal.

Ship What You Can Demonstrate

Can I name the domains where my framework has production evidence and the domains where it does not?

Am I willing to explicitly disclaim the domains I cannot demonstrate?

Do I keep deferring the hard problems to the next revision, betting they will be easier later?

A framework that ships for three domains with production evidence and explicitly says “we do not yet serve the fourth domain” is stronger than a framework that claims four domains and has evidence for three. The disclaimer is not weakness. It is honesty. And honesty is what the committee needs most from its architects.

Every deferral is a bet that the problem will be easier in the next revision. Some problems are not easier later. Some problems are structural - they arise from the framework's foundational decomposition and cannot be resolved without changing the decomposition itself. If the same problem survives three revisions, it is not a bug awaiting a fix. It is a boundary awaiting a name.

Scope the claim. Ship the core. Let the domains you cannot yet demonstrate be the motivation for the next proposal, not the unsubstantiated promise in this one.

Know Yourself

What if I am wrong?

The guidance in this paper is technical in form. This closing is not. It is about the human being behind the proposal.

We are emotional creatures. We cannot separate the human from the engineer. The proposal that consumed three years of our life is not just a document - it is identity. When someone challenges the proposal, they challenge us. When the committee questions the scope, it questions our judgment. When a domain expert says “this does not work in my domain,” we hear “you are not as good as you think you are.”

That is the moment where humility matters most. Not the easy humility of crediting collaborators in the acknowledgments section. The hard humility of recognizing that our emotions are driving our defense of a technical position. The hardest question in the committee is not “is my framework correct?” It is “am I defending this because the evidence supports it, or because I need it to be true?”

We have to be constantly on guard for when our desires are steering our engineering. The ambition that drives great work is the same ambition that prevents us from seeing its limits. The conviction that carried a proposal through ten revisions is the same conviction that makes the eleventh revision carry an unresolved problem rather than disclaim a domain. The passion that fills a room with allies is the same passion that empties a seat when a dissenter concludes that the room is no longer listening.

To know yourself - your motivations, your blind spots, your emotional investment in your own work - is the best gift you can give to the committee, to the standard, and to the users who depend on both.

This paper is a companion to “Big Claims Require Big Evidence,” which proposes process safeguards for the committee. Process catches the failures we cannot catch ourselves. But the failures we can catch ourselves - the scope claim we know is unsubstantiated, the criticism we know is valid, the domain we know we have not demonstrated - those are ours to own. No process can substitute for an author who is honest with themselves.

Big papers require big humility. The standard deserves both.

Acknowledgments

The following individuals were asked to review a draft of this paper. If your name appears below and you consent to being listed in the published version, please let the author know. Names will be removed before publication unless consent is given.

- Dietmar Kühl (Reviewed, Accepted, Declined)
- Mark Hoemmen (Reviewed, Accepted, Declined)